



UNIVERSIDADE ESTADUAL DO PIAUÍ
CAMPUS PROF. ALEXANDRE ALVES DE OLIVEIRA
CURSO SUPERIOR DE TECNOLOGIA EM SISTEMAS DE COMPUTAÇÃO

PROJETO DE PESQUISA

**PRÉ-PROCESSAMENTO DE BAIXO CUSTO PARA MITIGAÇÃO DE INVERSÕES:
OTIMIZANDO A EFICIÊNCIA DE ALGORITMOS QUADRÁTICOS**

Samuel da Penha Nascimento
Autor

Francisco das Chagas Rocha
Orientador

Parnaíba – Piauí
2026

SAMUEL DA PENHA NASCIMENTO

PRÉ-PROCESSAMENTO DE BAIXO CUSTO PARA MITIGAÇÃO DE INVERSÕES:
OTIMIZANDO A EFICIÊNCIA DE ALGORITMOS QUADRÁTICOS

Projeto de Pesquisa apresentado ao Curso Superior de Tecnologia em Sistemas de Computação como requisito parcial para o cumprimento do componente curricular obrigatório Trabalho de Conclusão de Curso.

Orientador: Prof. Mestre Francisco das Chagas Rocha.

A ordenação de dados permanece como uma das operações fundamentais e mais recorrentes na Ciência da Computação, servindo de base para a eficiência de sistemas de busca, processamento de bancos de dados e computação gráfica (Knuth, 1973; Sedgewick; Wayne, 2011). Embora algoritmos com complexidade de tempo média e de pior caso assintoticamente superiores, como o *MergeSort* e o *QuickSort*, sejam preferíveis para o tratamento de grandes volumes de dados, métodos de complexidade quadrática $O(n^2)$ tais como *InsertionSort*, *SelectionSort* e *BubbleSort*, retêm relevância em cenários operacionais específicos (Cormen et al., 2022).

Esta persistência está atrelada à simplicidade estrutural desses algoritmos, ao baixo consumo de memória adjacente e à eficiência prática demonstrada em conjuntos de dados de pequena escala ou parcialmente ordenados (Cormen et al., 2022). O *InsertionSort*, por exemplo, exibe um fator constante menor quando comparado ao *QuickSort*, o que resulta em maior velocidade de execução para vetores reduzidos, tipicamente compreendidos entre 10 e 50 elementos (Cormen et al., 2022). Em ambientes computacionais caracterizados pela severa restrição de recursos de *hardware*, tais propriedades transformam as abordagens quadráticas em alternativas viáveis frente a códigos teoricamente mais eficientes (Cormen et al., 2022).

A literatura demonstra que a eficiência prática de um algoritmo não é determinada exclusivamente por sua classe de complexidade assintótica, sendo fortemente influenciada pela sensibilidade do método às propriedades intrínsecas da sequência de entrada (LaMarca; Ladner, 1999). Nesse contexto, o conceito de inversão — definido por pares de elementos $A[i], A[j]$ tais que $i < j$ e $A[i] > A[j]$ — constitui o principal indicador do grau de desordem de um arranjo (LaMarca; Ladner, 1999). O desempenho físico do algoritmo é condicionado pela forma como os acessos aos dados interagem com a hierarquia de memória, visto que arranjos com menor índice de inversões minimizam a necessidade de operações de permuta de longa distância (LaMarca; Ladner, 1999).

O nível de ordenação prévia (*presortedness*) de um vetor pode ser quantificado analiticamente pelo número de inversões existentes, permitindo que algoritmos adaptativos apresentem um desempenho otimizado à medida que a desordem inicial decresce (Mannila, 1984). Um algoritmo de ordenação é classificado como ótimo em relação à pré-ordenação se o seu tempo de execução decresce de forma proporcional à redução da medida de desordem (Mannila, 1984). Por conseguinte, a formulação de estratégias voltadas à identificação e

mitigação prévia de inversões atua diretamente na atenuação dos custos operacionais que afetam os métodos quadráticos em instâncias altamente desordenadas (Mannila, 1984).

A redução sistemática do grau de entropia dos dados pode aproximar o comportamento dinâmico de determinados algoritmos quadráticos de padrões de execução observados em cenários de alta eficiência (Estivill-Castro; Wood, 1992). Evidencia-se, assim, a relevância da investigação de técnicas adaptativas orientadas ao aproveitamento das características estruturais dos dados de entrada, sobretudo em aplicações onde o tempo de resposta prático sobrepuja o valor da análise assintótica clássica de pior caso (Estivill-Castro; Wood, 1992). A eficácia de um algoritmo correlaciona-se à sua capacidade de explorar qualquer ordenação parcial preexistente, evitando o desperdício de ciclos de CPU em dados próximos de seu estado final (Estivill-Castro; Wood, 1992).

A introdução de etapas de pré-processamento em rotinas de ordenação também atende à demanda de minimização dos custos operacionais vinculados à movimentação física de registros e à alocação de memória (Bentley; McIlroy, 1993). Abordagens adaptativas consolidadas em algoritmos híbridos contemporâneos, como o *TimSort*, atestam a importância da detecção prévia de subsequências ordenadas (*runs*) como mecanismo de otimização para fluxos de dados reais (Peters, 2011). Dados gerados em sistemas produtivos frequentemente contêm padrões ordenados ascendentes ou descendentes que podem ser combinados de forma eficiente, distanciando-se de distribuições puramente aleatórias (Peters, 2011).

O estudo de mecanismos de pré-processamento projetados para reduzir a desordem inicial em algoritmos quadráticos apresenta relevância teórica e aplicada (Skiena, 2020). Em domínios técnicos restritivos, como sistemas embarcados e aplicações de tempo real, parâmetros como previsibilidade de execução, baixo consumo de memória e simplicidade de implementação frequentemente assumem maior relevância do que a eficiência assintótica de pior caso (Tanenbaum; Bos, 2015). Nesse contexto, determinados algoritmos de ordenação quadrática apresentam características que podem favorecer sua utilização, tais como estruturas de controle simples, reduzido overhead de memória e comportamento de execução mais facilmente analisável quando comparados a abordagens mais complexas baseadas em divisão e conquista ou em mecanismos de escolha dinâmica de pivôs (Hennessy; Patterson, 2019).

2. Problemática

A ordenação de dados constitui uma operação elementar e ubíqua no escopo da engenharia de *software* e da infraestrutura de computação (Knuth, 1973; Sedgewick; Wayne, 2011). Contudo, a eficiência prática dos algoritmos clássicos sofre tensões decorrentes do descompasso entre suas classes de complexidade assintótica e as demandas contemporâneas de processamento em larga escala (Cormen et al., 2022). Métodos de natureza quadrática, a exemplo do *BubbleSort*, *SelectionSort* e *InsertionSort*, possuem complexidade temporal de pior caso e caso médio delimitada pela função $O(n^2)$ (Cormen et al., 2022). Em cenários de grandes volumes de dados, essa progressão geométrica gera um consumo proibitivo de ciclos de CPU, restringindo a aplicação desses métodos quando comparados a paradigmas baseados em divisão e conquista, como o *MergeSort* e o *QuickSort*, cujos limites de desempenho se situam em $O(n \log n)$ (Cormen et al., 2022).

Os gargalos associados a esses algoritmos, no entanto, estendem-se além da contagem abstrata de instruções teóricas (LaMarca; Ladner, 1999). O desempenho real de execução é fortemente condicionado pelas interações operacionais entre a lógica do algoritmo e o custo de movimentação física de registros na estrutura de memória (LaMarca; Ladner, 1999). Algoritmos quadráticos, malgrado sua simplicidade estrutural e reduzido overhead de espaço (*in-place*), geram um volume elevado de operações de permuta e deslocamentos sucessivos quando o vetor apresenta alta entropia (Ziviani, 2021). Esse comportamento penaliza a latência temporal e o rendimento computacional global em arquiteturas modernas (Drepper, 2007; Ziviani, 2021). Assim, parâmetros como a distribuição das inversões e o custo de tráfego de dados na manipulação dos arranjos configuram-se como elementos determinantes para o tempo de execução físico, muitas vezes sobrepujando a análise nominal de comparações lógicas isoladas (Skiena, 2020).

Um fator crítico reside na insensibilidade adaptativa intrínseca a esses algoritmos diante da entropia e da configuração inicial do vetor de entrada (Mannila, 1984). A eficiência de um processo de ordenação está vinculada ao *presortedness* da sequência, que é mensurado de forma direta pela quantidade de inversões remanescentes (Mannila, 1984). O núcleo do problema manifesta-se no desperdício de recursos computacionais promovido por algoritmos não adaptativos que processam os dados ignorando o nível de ordenação original da entrada (Estivill-Castro; Wood, 1992). Paralelamente, rotinas como o *InsertionSort*, embora eficientes para sequências quase totalmente ordenadas, experimentam uma degradação severa em sua performance quando o índice de inversões é elevado, devido ao alto custo operacional

decorrente do deslocamento físico e sucessivo de elementos na estrutura do vetor (Cormen et al., 2022).

Apesar de tais deficiências de desempenho, a preservação de métodos quadráticos é demandada em ecossistemas computacionais restritos, tais como sistemas embarcados e aplicações de tempo real rígido (*Hard Real-Time*), nos quais a previsibilidade do tempo de execução e o reduzido uso de memória (*footprint*) constituem requisitos mandatórios (Tanenbaum; Bos, 2015). A lacuna de pesquisa evidencia-se na necessidade de mitigar o custo quadrático dessas abordagens sem comprometer a simplicidade que as torna viáveis nesses ambientes (Tanenbaum; Bos, 2015). Embora a literatura apresente estudos sobre medidas de presortedness, algoritmos adaptativos e estratégias de pré-ordenação, permanece relevante investigar em que medida mecanismos de pré-processamento de baixo custo podem contribuir para a melhoria do desempenho prático de algoritmos quadráticos de ordenação em diferentes cenários de entrada (Peters, 2011).

O problema fundamental mapeado na literatura consiste na necessidade de técnicas de pré-processamento focadas na redução do número de inversões e no aproveitamento de propriedades estruturais da entrada, cujo custo computacional obedeça à relação (Skiena, 2020): $C_{pre} + C_{sort} < C_{original}$. O desafio central repousa na superação do gargalo provocado pela movimentação excessiva de dados, aperfeiçoando a resposta temporal em cenários onde a análise puramente assintótica não reflete a realidade da execução física no *hardware* (Skiena, 2020).

A partir do cenário problemático delineado pela literatura, estabelece-se a seguinte questão de pesquisa: **Em que medida a implementação de uma etapa de pré-processamento voltada à redução de inversões impacta o desempenho prático de algoritmos de ordenação quadráticos?**

3. Objetivos

Objetivo Geral

Propor, implementar e avaliar um algoritmo de pré-processamento que reduz o número de inversões de uma entrada e que a soma do pré-processamento com a ordenação seja mais eficiente que o algoritmo de ordenação puro.

Objetivos Específicos:

1. Analisar teoricamente como o número de inversões afeta o tempo de execução dos algoritmos quadráticos alvos (*InsertionSort*, *SelectionSort* e *BubbleSort*), estabelecendo a base teórica para o pré-processamento.
2. Desenvolver o design lógico e matemático do algoritmo simétrico de baixo custo, almejando que sua complexidade assintótica de tempo seja inferior a $O(n \log n)$ e que ele opere de forma *in-place*.
3. Codificar o algoritmo de pré-processamento e os algoritmos quadráticos em Rust, garantindo uma implementação eficiente e modular para a coleta de dados de tempo de execução.
4. Mensurar o tempo físico de execução da abordagem híbrida (pré-processamento + ordenação) em cenários com diferentes volumetrias de dados e graus de ordenação prévia (*presortedness*), incluindo casos de vetores invertidos e parcialmente ordenados.
5. Demonstrar estatisticamente os cenários onde a inequação $C_{pre} + C_{sort} < C_{original}$ é verdadeira, identificando os limites de tamanho de entrada (n) onde o pré-processamento deixa de ser vantajoso devido ao seu próprio *overhead*.

O desenvolvimento de algoritmos eficientes constitui um dos principais desafios da Ciência da Computação, especialmente diante do crescimento contínuo do volume de dados processados pelos sistemas computacionais contemporâneos. Embora algoritmos de ordenação com melhor desempenho assintótico sejam amplamente empregados em aplicações de larga escala, métodos de complexidade quadrática ainda possuem relevância devido à sua simplicidade estrutural, facilidade de implementação e utilização em contextos educacionais, sistemas legados e aplicações nas quais o tamanho da entrada é controlado (Cormen et al., 2022; Sedgewick; Wayne, 2011).

Nesse contexto, torna-se relevante investigar estratégias complementares capazes de reduzir o custo operacional desses algoritmos sem necessariamente substituir sua estrutura fundamental. Uma possibilidade consiste na aplicação de técnicas de pré-processamento de baixo custo computacional, capazes de modificar características da entrada antes da execução do algoritmo principal, reduzindo fatores que contribuem diretamente para o aumento do número de operações realizadas.

Dessa forma, este referencial teórico organiza-se em cinco blocos conceituais interdependentes. Inicialmente, são apresentados os fundamentos dos algoritmos de ordenação e da análise de complexidade computacional, estabelecendo as métricas utilizadas para avaliar o desempenho dos métodos analisados (Knuth, 1973; Szwarcfiter; Markenzon, 2009). Em seguida, são discutidos os algoritmos quadráticos de ordenação, com destaque para suas limitações computacionais em cenários de crescimento significativo da entrada (Drozdek, 2021; Ziviani, 2021). Posteriormente, são abordadas estratégias tradicionais de otimização desses algoritmos, evidenciando a necessidade de mecanismos auxiliares de melhoria de desempenho. Na sequência, apresenta-se o conceito de inversões em arranjos, utilizado como medida quantitativa do grau de desordem dos dados de entrada e como indicador do esforço computacional exigido por algoritmos baseados em comparações locais (Cormen et al., 2022; Knuth, 1973). Por fim, discute-se o pré-processamento algorítmico de baixo custo como estratégia para reorganizar a estrutura dos dados e potencializar o desempenho das etapas posteriores de ordenação.

4.1 Algoritmos de Ordenação e Complexidade Computacional

A ordenação de dados consiste no processo de reorganização de uma sequência de elementos segundo determinado critério de comparação, como ordem crescente, decrescente ou alfabética, representada na Figura 1, permitindo operações posteriores de busca, análise e manipulação de informações de maneira mais eficiente (Cormen et al., 2022). Essa operação representa uma das tarefas fundamentais da computação, estando presente em diversos sistemas que necessitam organizar grandes volumes de dados.

Figura 1 - Representação de um Algoritmo de Ordenação



Fonte: autoria própria.

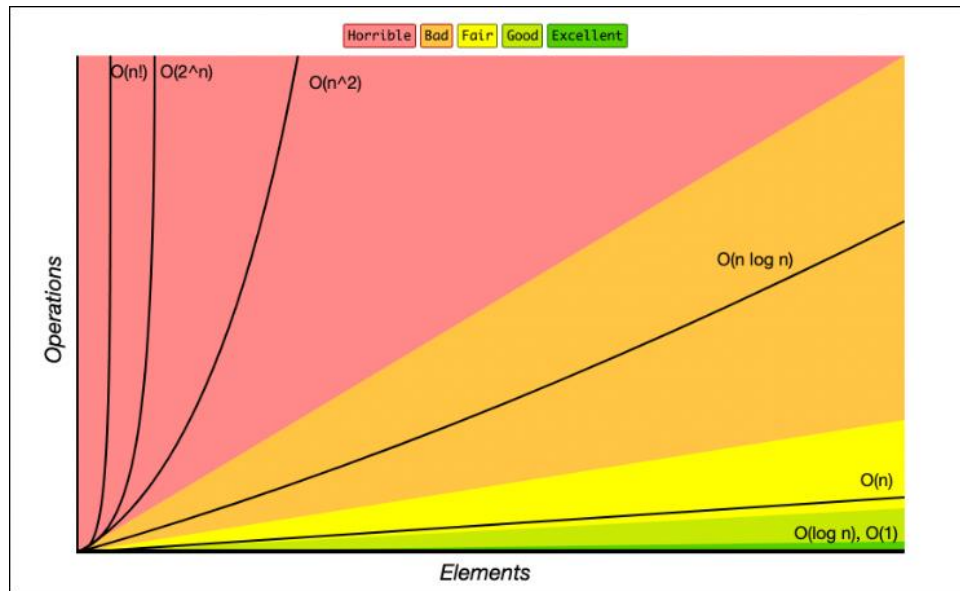
Com o aumento da quantidade de informações manipuladas pelos sistemas computacionais modernos, a eficiência dos algoritmos de ordenação tornou-se um fator determinante para o desempenho global das aplicações. Assim, diferentes métodos de ordenação são avaliados não apenas pela correteza dos resultados produzidos, mas também pela quantidade de recursos computacionais necessários durante sua execução (Sedgewick; Wayne, 2011).

A análise de complexidade computacional constitui uma ferramenta utilizada para estimar o comportamento dos algoritmos conforme ocorre o crescimento do tamanho da entrada. Entre as formas mais utilizadas de representação dessa análise destaca-se a Notação Big-O, responsável por descrever a ordem de crescimento do número de operações dominantes realizadas por um algoritmo em função do parâmetro de entrada n (Knuth, 1973).

Essa análise considera diferentes cenários de execução, conforme a Figura 2. O melhor caso representa a configuração de entrada que conduz ao menor número de operações; o pior

caso corresponde à situação que exige o maior esforço computacional possível; enquanto o caso médio representa uma estimativa do comportamento esperado considerando distribuições típicas das entradas analisadas (Szwarcfiter; Markenzon, 2009).

Figura 2 - Classificações da Notação Big-O



Fonte: <https://www.bigocheatsheet.com/>

4.2 Algoritmos Quadráticos ($O(n^2)$)

Algoritmos que possuem uma complexidade de tempo de execução classificada como quadrática, denotada por $O(n^2)$, apresentam um crescimento proporcional ao quadrado do tamanho do conjunto de dados de entrada, o que significa que duplicar o volume de elementos quadruplica o tempo necessário para processá-los (Ziviani, 2021). Essa característica de crescimento acentuado faz com que tais métodos sofram uma degradação severa de desempenho, tornando-se computacionalmente inviáveis quando aplicados a grandes conjuntos de informações (Cormen et al., 2022).

Entre as abordagens clássicas que compartilham essa taxa de crescimento, destacam-se o *BubbleSort*, o *SelectionSort* e o *InsertionSort* (Drozdek, 2021). O *BubbleSort* opera por meio de varreduras repetidas comparando e trocando elementos adjacentes fora de ordem; o *SelectionSort* baseia-se na busca constante do menor elemento restante para posicioná-lo no início; e o *InsertionSort* constrói a sequência final inserindo progressivamente cada elemento em sua posição correta (Goodrich; Tamassia; Goldwasser, 2014). Embora essas estruturas se destaquem pela simplicidade de implementação e facilidade de compreensão didática, o alto custo computacional gerado pelos laços de repetição aninhados limita estritamente sua utilidade prática a arranjos de dimensões reduzidas (Sedgewick; Wayne, 2011).

4.3 Estratégias de Otimização Para Algoritmos Quadráticos

Diversas abordagens foram propostas para melhorar o desempenho de algoritmos quadráticos sem alterar completamente sua estrutura original. No caso do BubbleSort, por exemplo, uma otimização tradicional consiste na inclusão de mecanismos capazes de identificar antecipadamente situações nas quais nenhuma troca foi realizada durante determinada passagem, permitindo interromper a execução antes do processamento completo das comparações restantes (Cormen et al., 2022).

Entretanto, tais otimizações geralmente atuam sobre o funcionamento interno do algoritmo, mantendo a dependência direta entre o desempenho e a organização inicial dos dados. Dessa forma, estratégias baseadas na preparação da entrada representam uma possibilidade complementar, pois buscam reduzir previamente características responsáveis pelo aumento do custo operacional.

Nesse sentido, compreender o grau de desordem presente nos dados torna-se fundamental para desenvolver mecanismos capazes de antecipar ou reduzir o esforço necessário pelos algoritmos de ordenação.

4.4 O Conceito de Inversões em Arranjos (Arrays)

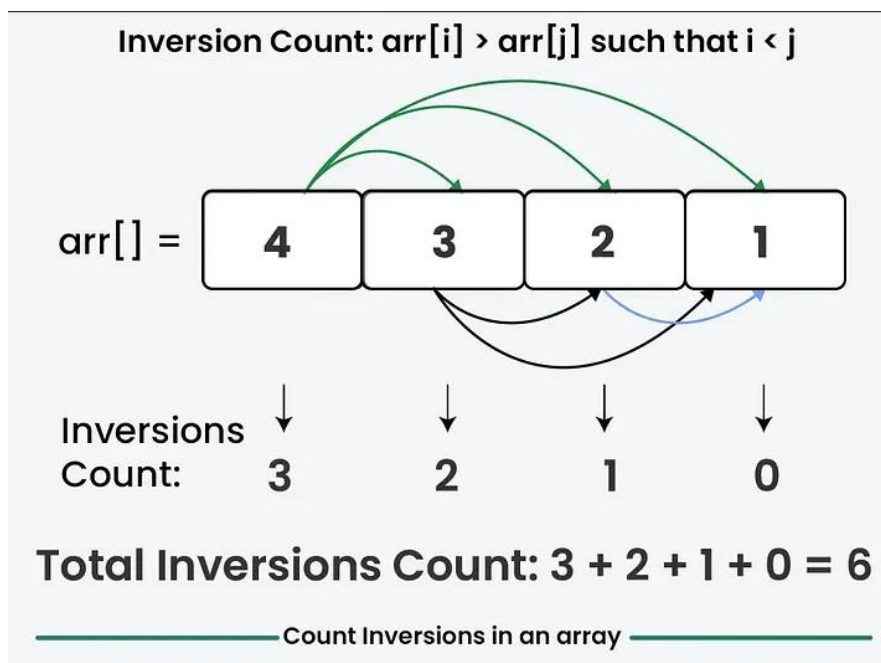
Em uma sequência de elementos, uma inversão é formalmente identificada sempre que um par de índices (i, j) satisfaz a condição de que $i < j$, conforme demonstrado na Figura 3, mas o valor contido na posição anterior é estritamente maior que o valor na posição posterior (Knuth, 1973). Esse fenômeno serve como uma métrica quantitativa do nível de desorganização ou desordem presente em um arranjo, indicando o quão distante a estrutura está de seu estado ideal completamente ordenado (Cormen et al., 2022).

A quantidade de inversões existentes em uma sequência possui relação direta com algoritmos de ordenação baseados em comparações locais, especialmente aqueles que utilizam trocas entre elementos adjacentes. No caso específico do *BubbleSort*, cada troca realizada entre elementos vizinhos fora de ordem reduz exatamente uma inversão da sequência, fazendo com que o número inicial de inversões represente uma estimativa do esforço necessário para ordenar completamente o conjunto de dados (Cormen et al., 2022).

Dessa forma, a quantidade de inversões presentes em um arranjo pode ser compreendida como um indicador direto da dificuldade que um algoritmo de ordenação baseado em comparações locais encontrará durante sua execução. Entradas contendo uma elevada quantidade de inversões exigem maior número de comparações e operações de troca, uma vez que o algoritmo precisa realizar mais movimentações para aproximar progressivamente os

elementos de suas posições corretas. Em contrapartida, sequências que apresentam determinado grau de ordenação prévia possuem menor quantidade de inversões e, conseqüentemente, demandam menos operações para alcançar o estado final ordenado.

Figura 3 - Contagem de Inversões em um Arranjo



Fonte: <https://medium.com/%40alexey.medvecky/finding-inversions-in-an-array-algorithm-and-optimal-implementation-in-c-a13577a865c8>

4.5 Pré-processamento de Dados

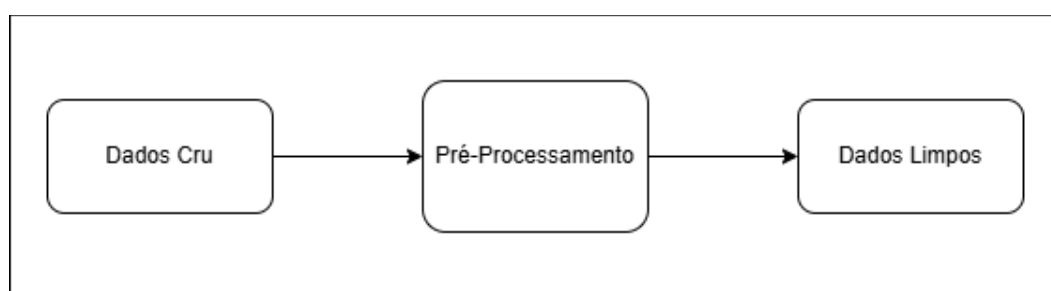
O pré-processamento de dados constitui uma fase preliminar do fluxo de computação na qual as informações originais são submetidas a transformações, filtrações ou reorganizações estruturais antes da execução do algoritmo principal de destino, conforme a Figura 4 (Han; Kamber; Pei, 2012). O propósito fundamental dessa etapa é preparar a base de dados de modo a facilitar o processamento subsequente, reduzindo a complexidade do problema ou otimizando as condições para que os métodos principais operem com maior eficácia (Tan; Steinbach; Kumar, 2006).

Para que a introdução de uma etapa de pré-processamento seja considerada vantajosa do ponto de vista da eficiência global, ela deve obedecer estritamente à lógica do baixo custo computacional, manifestando-se idealmente por meio de uma complexidade linear, ou $O(n)$ (Aho; Hopcroft; Ullman, 1974). Caso o consumo de tempo e de recursos dispendido na fase de preparação seja excessivo, o ganho de velocidade obtido na etapa de ordenação subsequente

será anulado, resultando em um balanço de desempenho global desfavorável ou ineficaz para o sistema (Ziviani, 2021).

Dessa maneira, considerando que o número de inversões representa uma medida da desordem inicial de um arranjo e influencia diretamente o esforço requerido por algoritmos baseados em trocas locais, torna-se possível investigar técnicas preliminares capazes de reduzir essa característica antes da ordenação. Essa abordagem fundamenta a hipótese de que um mecanismo de pré-processamento de baixo custo pode contribuir para a melhoria do desempenho global de algoritmos quadráticos em determinados cenários de entrada.

Figura 4 - Fluxo do Pré-Processamento



Fonte: autoria própria.

4.6 Trabalhos Relacionados

O objetivo do trabalho de Mannila (1984) foi formalizar e estabelecer critérios matemáticos para quantificar o quão ordenado um vetor de entrada se encontra antes de ser submetido a uma rotina de ordenação definitiva. Para isso, o autor introduziu e utilizou o conceito de *Measures of Presortedness* (Medidas de Pré-ordenação), mapeando diferentes métricas de desorganização, dentre as quais se destaca a função $Inv(X)$, que contabiliza o número exato de inversões em uma sequência. Como resultados alcançados, o estudo provou matematicamente que o número de inversões é o indicador ideal para prever o limite inferior do custo computacional de algoritmos baseados em comparações locais, fornecendo a base teórica para que transformações preliminares pudessem ser avaliadas de acordo com a sua capacidade de mitigar essas inversões na entrada.

O trabalho de Hwang, Yang e Yeh (2000) buscou analisar o impacto prático e o comportamento do caso médio de pipelines de computação que utilizam algoritmos de pré-processamento (denominados *presorting*) antes da ordenação final. Os autores utilizaram modelagem estatística assintótica e simulações analíticas para avaliar estratégias de reestruturação de dados que operam de forma preliminar ao algoritmo de destino, focando especificamente na redução da estatística de inversão do vetor. Os resultados alcançados

demonstraram que a introdução de uma etapa de pré-processamento, desde que mantida estritamente em complexidade linear ($O(n)$), reduz drasticamente o volume de inversões iniciais, alterando positivamente o comportamento assintótico das rotinas subsequentes e validando a viabilidade econômica desse modelo de arquitetura híbrida.

A pesquisa de Estivill-Castro e Wood (1992) teve como objetivo investigar e delimitar o desempenho de algoritmos de ordenação adaptativos (*adaptive sorting*) quando aplicados a conjuntos de dados que já passaram por algum nível de organização prévia. A metodologia aplicada consistiu na análise teórica e em testes empíricos do comportamento de métodos clássicos, com ênfase no *InsertionSort*, avaliando como o tempo de execução varia em função do número residual de inversões (I) contidas no arranjo. Os resultados alcançados comprovaram que o custo operacional do *InsertionSort* é diretamente proporcional a $O(n + I)$; dessa forma, os autores demonstraram empiricamente que, se um mecanismo anterior for capaz de mitigar a quantidade de inversões para um patamar linear, o algoritmo quadrático deixa de manifestar sua degradação severa $O(n^2)$ e passa a executar a etapa principal em tempo linear, viabilizando o uso de estruturas simples em cenários de grande escala.

Diferenciando-se das abordagens mapeadas, o presente projeto de pesquisa propõe um algoritmo de Pré-processamento Simétrico que atua na mitigação direta de inversões em tempo linear, preenchendo a lacuna de otimização de baixo custo para algoritmos quadráticos.

A presente pesquisa é classificada, quanto à sua natureza, como aplicada, uma vez que busca gerar conhecimentos voltados para a solução de um problema específico da ciência da computação e da análise de algoritmos, relacionado à otimização do tempo de execução de rotinas clássicas de ordenação. Segundo Gil (2017), a pesquisa aplicada caracteriza-se pela produção de conhecimentos com finalidade prática, direcionados à resolução de problemas concretos. Nesse contexto, o estudo propõe o desenvolvimento e a validação de um algoritmo simétrico de pré-processamento de baixo custo, capaz de mitigar o impacto de inversões em estruturas de dados e acelerar a eficiência de métodos quadráticos, contribuindo diretamente para o campo da engenharia de software e da otimização de sistemas computacionais.

Quanto aos objetivos, a pesquisa é classificada como explicativa e experimental. A característica explicativa decorre da investigação das relações de causa e efeito existentes entre a quantidade quantitativa de inversões (desordem atômica de um vetor) e a consequente degradação da curva temporal dos algoritmos *InsertionSort*, *SelectionSort* e *BubbleSort*. Conforme Prodanov e Freitas (2012), pesquisas explicativas buscam identificar os fatores que determinam ou contribuem para a ocorrência dos fenômenos estudados. Por sua vez, a natureza experimental manifesta-se na manipulação controlada de variáveis independentes (volumetria dos dados e graus de ordenação prévia ou *presortedness*) para observar e mensurar estatisticamente os efeitos sobre a variável dependente (tempo físico de execução da abordagem híbrida proposta).

Quanto à abordagem do problema, a pesquisa caracteriza-se como quantitativa. O método baseia-se na coleta e análise de dados numéricos obtidos por meio de testes de desempenho (*benchmarks*) após o desenvolvimento do algoritmo de pré-processamento. Esse modelo permite isolar variáveis e mensurar o tempo de execução e o comportamento dos algoritmos sob diferentes cenários. A escolha é adequada, pois o objetivo consiste em quantificar a eficiência do método proposto e testar as hipóteses por meio de dados métricos, gerando tabelas, gráficos e estatísticas comparativas que demonstrem a real vantagem do algoritmo frente aos métodos originais de ordenação.

A pesquisa será desenvolvida em etapas. Inicialmente, será realizada uma revisão bibliográfica aprofundada sobre complexidade assintótica, medidas de *presortedness*, contagem e propriedades matemáticas de inversões em permutações, além da análise tradicional de pior caso, caso médio e melhor caso dos algoritmos quadráticos alvos. Essa etapa fornecerá o embasamento teórico necessário para o design lógico e matemático do algoritmo simétrico.

Posteriormente, será realizada a etapa de especificação e design formal do algoritmo de pré-processamento. Nesta fase, serão modeladas as equações de transição de estados e conduzidas análises matemáticas destinadas a determinar sua complexidade assintótica de tempo e espaço, bem como verificar se o algoritmo atende aos requisitos de operação in-place. Os resultados dessa análise fornecerão a fundamentação teórica necessária para caracterizar formalmente o comportamento computacional da solução proposta.

Para o desenvolvimento do ambiente computacional e a execução dos experimentos, será utilizada a linguagem de programação Rust, amplamente reconhecida na literatura técnica por sua segurança de memória sem *Garbage Collector*, previsibilidade de desempenho em sistemas de tempo real e controle granular de hardware. Entre os recursos da linguagem previstos estão o sistema de tipos estático e linear para garantir a ausência de alocações secundárias indesejadas, além de ferramentas nativas de *benchmarking* de alta precisão (como a biblioteca *criterion*), específicas para mitigar o impacto de otimizações agressivas do compilador (como o *dead code elimination*) na coleta de telemetria.

Na etapa de simulação e geração de dados, serão construídos geradores sintéticos de vetores capazes de parametrizar rigorosamente o tamanho das entradas (n) e os coeficientes de inversão. Serão modelados cenários críticos de teste, contemplando vetores perfeitamente ordenados (melhor caso), vetores integralmente invertidos (pior caso), e vetores parcialmente ordenados contendo ruídos estocásticos controlados de desordem.

A validação do desempenho computacional será realizada por meio de testes empíricos de tempo físico de execução (tempo de CPU e *wall-clock time*). Para cada cenário experimental, serão realizadas 30 execuções independentes, de modo a reduzir o impacto de variações ocasionais do ambiente de execução. Os resultados obtidos serão consolidados por meio da média aritmética dos tempos observados. Os experimentos confrontarão o tempo gasto pelos algoritmos clássicos isolados com o tempo total consumido pela abordagem híbrida (tempo de pré-processamento somado ao tempo de ordenação residual), permitindo a validação direta da inequação de eficiência $C_{pre} + C_{sort} < C_{original}$.

Por fim, os resultados obtidos serão analisados de forma quantitativa, plotando-se as curvas empíricas de desempenho em função do tamanho da entrada e do grau de inversão. Espera-se que a análise identifique com precisão geométrica os pontos de inflexão (*overhead*) a partir dos quais o custo do pré-processamento deixa de ser vantajoso, fornecendo um mapeamento claro das fronteiras de eficiência do algoritmo simétrico proposto.

6. Resultados Esperados

1. Mapeamento Teórico do Impacto das Inversões: A consolidação de uma base teórica e analítica detalhada que demonstre matematicamente como a variação quantitativa de inversões determina a curva de degradação temporal dos algoritmos *InsertionSort*, *SelectionSort* e *BubbleSort*, servindo de fundamentação lógica para a necessidade do pré-processamento.
2. Formalização e Design do Algoritmo Simétrico: A especificação lógica, formal e matemática do algoritmo de pré-processamento simétrico proposto, acompanhada da determinação da complexidade assintótica do algoritmo proposto e que seu consumo de memória adjacente é nulo (caracterizando uma operação estritamente *in-place*, ou $O(1)$ espaço).
3. Ambiente de Testes Modular em Rust: O desenvolvimento de um ecossistema de software robusto, eficiente e modular codificado em linguagem Rust, contendo as implementações otimizadas tanto das abordagens quadráticas isoladas quanto do algoritmo de pré-processamento, estruturado especificamente para a coleta precisa de telemetria e métricas de execução.
4. Dataset e Relatórios de Mensuração Física: A obtenção de um conjunto de dados empíricos detalhados sobre o tempo físico de execução de cada abordagem, mapeando o comportamento dinâmico do sistema híbrido frente a variações controladas de volumetria de dados (n) e diferentes coeficientes de *presortedness* (incluindo os cenários críticos de vetores integralmente invertidos e parcialmente ordenados).
5. Validação Estatística e Fronteiras de Eficiência: A demonstração estatística e a delimitação clara dos cenários operacionais onde a inequação $C_{pre} + C_{sort} < C_{original}$ é válida, identificando com precisão o ponto de inflexão (*overhead*) a partir do qual o tamanho da entrada (n) ou o nível de ordenação prévia tornam o pré-processamento desvantajoso frente aos métodos originais.

7. Recursos e Equipamentos

Esta pesquisa não recebeu financiamento de nenhuma fonte externa, tendo sido conduzida de forma independente pelos pesquisadores. O único equipamento utilizado foi um computador pessoal, empregado tanto para a revisão da literatura existente sobre o tema quanto para o desenvolvimento da solução e a gestão dos dados coletados durante o estudo.

Não foram utilizadas tecnologias específicas além das necessárias para o acesso à Internet, o que inclui programas de processamento de texto para a redação da pesquisa e softwares de planilha para a organização dos dados coletados.

A pesquisa foi conduzida sem a colaboração de outras instituições de pesquisa ou empresas públicas e privadas. Todas as atividades, desde a coleta de dados até a análise e a redação, foram realizadas exclusivamente pelos pesquisadores.

8. Cronograma Geral

ATIVIDADES	Ano/Meses					
	2026					
	Jul	Ago	Set	Out	Nov	Dez
I – Desenvolvimento da Pesquisa Acerca do Impacto de Inversões	X					
II – Modelagem do Algoritmo de Pré-Processamento com Base na Pesquisa Realizada		X	X			
III – Desenvolvimento do Algoritmo Seguindo o Modelo Proposto			X	X		
IV - Escrita do Artigo Científico			X	X	X	
V – Realização de Testes e Obtenção de Resultados				X	X	
VI – Análise dos Dados Obtidos					X	
VII - Defesa do Trabalho de Conclusão de Curso						X

- AHO, Alfred V.; HOPCROFT, John E.; ULLMAN, Jeffrey D. **The design and analysis of computer algorithms**. Reading, Mass: Addison-Wesley Pub. Co, 1974.
- BENTLEY, Jon L.; MCILROY, M. Douglas. Engineering a sort function. **Software: Practice and Experience**, v. 23, n. 11, p. 1249–1265, nov. 1993.
- CORMEN, Thomas H. *et al.* **Introduction to algorithms**. Fourth edition ed. Cambridge, Massachusetts London, England: The MIT Press, 2022.
- DREPPER, Ulrich. What Every Programmer Should Know About Memory. *In*: 2007. Disponível em: <<https://api.semanticscholar.org/CorpusID:6440708>>
- DROZDEK, Adam. **Estrutura de dados e algoritmos em c++: tradução da 4ª edição norte-americana**. [S.l.]: Cengage Learning, 2021.
- ESTIVILL-CASTRO, Vladimir; WOOD, Derick. A survey of adaptive sorting algorithms. **ACM Computing Surveys**, v. 24, n. 4, p. 441–476, dez. 1992.
- GIL, Antonio Carlos. **Como elaborar projetos de pesquisa**. [S.l.]: Editora Atlas Ltda, 2017.
- GOODRICH, Michael T.; TAMASSIA, Roberto; GOLDWASSER, Michael H. **Data structures and algorithms in Java**. 6. ed ed. Hoboken, NJ: Wiley, 2014.
- HAN, Jiawei; KAMBER, Micheline; PEI, Jian. **Data mining: concepts and techniques**. 3rd ed ed. Burlington, MA: Elsevier, 2012.
- HENNESSY, John L.; PATTERSON, David A. **Computer architecture: a quantitative approach**. Sixth edition ed. Cambridge, Massachusetts: Elsevier, 2019.
- HWANG, Hsien-Kuei; YANG, Bo-Yin; YEH, Yeong-Nan. Presorting algorithms: An average-case point of view. **Theoretical Computer Science**, v. 242, n. 1–2, p. 29–40, jul. 2000.
- KNUTH, Donald Ervin. **The art of computer programming**. Second ed ed. Reading (Mass.) London Manila [etc.]: Addison-Wesley publ, 1973.
- LAMARCA, Anthony; LADNER, Richard E. The Influence of Caches on the Performance of Sorting. **Journal of Algorithms**, v. 31, n. 1, p. 66–104, abr. 1999.
- MANNILA, Heikki. Measures of presortedness and optimal sorting algorithms: Extended abstract. *In*: PAREDAENS, Jan (Org.). **Automata, Languages and Programming**. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer Berlin Heidelberg, 1984. v. 172 p. 324–336.
- PETERS, Tim. **Timsort description**. GitHub, 2011. Disponível em: <<https://github.com/python/cpython/blob/main/Objects/listsorth.txt>>. Acesso em: 30 maio. 2026
- PRODANOV, Cleber Cristiano. **Metodologia do trabalho científico: métodos e técnicas da pesquisa e do trabalho acadêmico**. [S.l.]: Universidade Feevale, 2012.
- SEDGEWICK, Robert; WAYNE, Kevin Daniel. **Algorithms**. 4th ed ed. Upper Saddle River: Addison-Wesley, 2011.
- SKIENA, Steven S. **The algorithm design manual**. Third edition ed. Cham: Springer, 2020.

SZWARCFITER, Jayme; MARKENZON, Lilian. **Estruturas de Dados e Seus Algoritmos**. [S.l.]: Ltc-Livros Tecnicos E Cientificos Editora Lda, 2009.

TAN, Pang-Ning; STEINBACH, Michael; KUMAR, Vipin. **Introduction to data mining**. 1st ed ed. Boston: Pearson Addison Wesley, 2006.

TANENBAUM, Andrew S.; BOS, Herbert. **Modern operating systems**. 4. ed ed. Boston: Prentice Hall, 2015.

ZIVIANI, Nivio. **Projeto De Algoritmos Com Implementações Em Java E C++**. [S.l.]: Cengage Learning, 2021.